

FluencyOS

The codebase, explained in plain English

Software Engineering Project · Jubair · Rafsan · Foyez

In one sentence: FluencyOS is a desktop app that helps you learn English from the films and books you already enjoy — you look up a word where you met it, it gets saved with that exact moment, and later you have to actually *use* it in conversation before the app considers it learned.

This document explains how the code is put together. It is written for someone who knows a little programming but has not seen this project before — a teammate joining, or a reviewer trying to understand what was built.

1. The big picture

The app is made of **three programs that run at the same time** on your computer and talk to each other. Think of it like a restaurant:

THE BUILDING

Electron — the desktop window itself. It opens the app window, owns the file system, and starts the other two pieces. Written in TypeScript.

THE DINING ROOM

React UI — everything you see and click. It is a website, but running inside the desktop window instead of a browser. TypeScript + React + Tailwind.

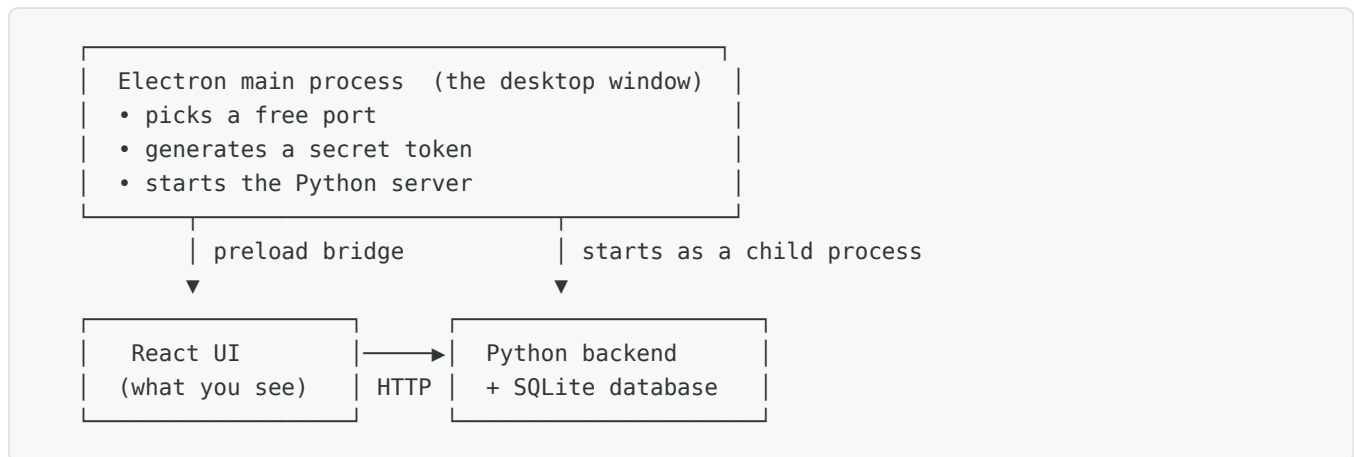
THE KITCHEN

Python backend — a small web server running privately on your own machine. It does the thinking and owns the database. FastAPI + SQLite.

Why is there a Python server inside a desktop app?

Because the interesting parts of this project — running a local AI language model, converting speech to text, generating speech — are all Python libraries. Rather than fight that, the app just runs a tiny Python web server in the background. It is **not** on the internet: it listens only on `127.0.0.1`, which means "this computer only". Nothing leaves the machine.

How the three pieces talk



2. What happens when you launch the app

- 1 You run `npm run dev`. A small script (`scripts/dev.mjs`) starts the UI build tool and then launches the Electron window.
- 2 Electron picks a **random free port number** and generates a **random secret token** (64 random characters, new every launch).
- 3 Electron starts the Python server as a background process, handing it the port, the token, and where to put the database file.
- 4 The Python server creates the database if it does not exist, then reports it is healthy. Electron keeps checking until it answers.
- 5 Only then does Electron open the window and load the React UI.
- 6 The UI asks Electron "where is the backend and what is the password?", then talks to it directly for everything after that.
- 7 When you close the window, Electron shuts the Python server down too, so nothing is left running.

The secret token — why it exists

A web server running on your computer could, in principle, be called by *any* other program on that computer. To prevent that, every single request must include the secret token that was generated at launch. Anything without it is rejected immediately. The token lives only in memory — it is never written to a file, never logged, and it changes every time you start the app.

3. A tour of the folders

```
fluency-os/  
├─ electron/      the desktop window and the bridge to the operating system  
├─ renderer/      everything you see – the React user interface  
├─ backend/       the Python server, the database, and the "thinking" code  
├─ scripts/       helper scripts that build and launch everything  
├─ build/         the app icon  
└─ docs/          this document
```

electron/ — the desktop shell

FILE	WHAT IT DOES
main/index.ts	Creates the app window and controls the app's life cycle (start up, shut down).
main/backend-process.ts	Starts, health-checks, and stops the Python server.
main/handshake.ts	Generates the secret token.
main/system-info.ts	Reads the real CPU core count and RAM size from the operating system.
main/ipc-handlers.ts	The list of things the UI is allowed to ask the operating system for.
preload/index.ts	The only door between the UI and the operating system (see below).

Why the "preload bridge" matters. The UI is essentially a web page, and web pages should never have direct access to your files. So the UI is completely sandboxed. It cannot read a file or run a command. It can only call four specific functions that the preload file exposes: get the backend address, get the hardware info, open a folder picker, and control the window buttons. Everything else is blocked by design.

renderer/ — the user interface

The UI is organised by **feature**, not by file type. Each screen lives in its own folder:

FOLDER	SCREEN
features/onboarding/	The 5-step first-run wizard.
features/shell/	The frame around everything: title bar, left sidebar, page header.
features/dashboard/	The home screen — what's due, streak, progress charts.
features/library/	"Learn by watching" — your video library and the add-content dialog.
features/player/	The video player with clickable subtitles and the word-lookup panel.
features/bookshelf/	"Learn by reading" — your books and reading goal.
features/reader/	The e-reader with highlights, AI lookups and text simplification.
features/vocabulary/	Your saved words list and the detail page for each word.
store/	The app's shared memory (explained below).
lib/apiClient.ts	One small helper that talks to the backend and attaches the secret token.
types/	Shared descriptions of the data shapes, so mistakes get caught while coding.

backend/ — the Python server

FILE	WHAT IT DOES
app/main.py	Starts the server, sets up the database, connects the routes.
app/security.py	Checks the secret token on every request.
app/db.py	Opens the database connection with the right safety settings.
app/migrations/	The database structure, and the code that creates/updates it.
app/routers/	The web addresses the UI can call (users, placement test, hardware check).
app/services/	The actual decision-making logic, kept separate so it can be tested.
app/models/	Descriptions of what each request and response must contain.
tests/	24 automated tests that check the logic still works.

4. How the UI remembers things

React on its own is forgetful — each piece of the screen only knows about itself. So the app uses three small shared "notebooks" (called stores) that any screen can read from:

STORE	REMEMBERS
appStore	Who is logged in, whether the backend is ready, and whether onboarding is finished. This is what decides if you see the wizard or the dashboard when you open the app.
onboardingStore	Everything you type during the 5-step wizard, plus the logic for saving each step to the backend.
shellStore	Which screen you are looking at, whether the sidebar is collapsed, dark or light theme, and which video/book/word you last opened.

How navigation works

There is no browser-style URL routing. The `shellStore` simply holds the name of the current screen (like `"dashboard"` or `"player"`), and `App.tsx` shows the matching component. Clicking a sidebar item just changes that name.

Some screens are "immersive" — the video player and the e-reader. When you open one, the sidebar automatically shrinks to icons only, so the content gets more room.

5. The design system

Every colour, font and spacing value in the app comes from a single file:

`renderer/src/styles/globals.css`. It defines the palette twice — once for dark mode, once for light mode — and the whole app switches instantly by changing one attribute on the page.

TOKEN	MEANING
--acc	The forest green (<code>#3E7C5A</code>) used for anything active or important.
--bg / --panel	The page background and the card background.
--tx / --tx2 / --tx3	Text in three strengths: main, secondary, faint.
--line / --line2	Borders, strong and subtle.

These were copied exactly from the visual mock-up that was designed before coding started, so the built app matches the design pixel for pixel. Tailwind is configured to point at these tokens, which is why the code says things like `bg-panel` and `text-tx3` instead of raw colour codes.

6. The database

The app uses **SQLite** — a database that is just a single file on your disk, with no separate program to install. It lives at `~/.config/fluencyos/fluencyos.db` on Linux.

TABLE	HOLDS
users	Your name, native language, target language, CEFR level, and when you finished onboarding.
user_settings	Your preferences: which AI model, daily goals, quiet hours, clip quality, and so on.
app_meta	Housekeeping: database version, install id, and a few temporary values.

Two safety settings are switched on for every connection: **WAL mode** (so reading and writing at the same time is safe) and **foreign keys** (so the database refuses to store rows that point at things that do not exist).

The database structure is created by `migrations/0001_init.sql`. The runner keeps a record of which migration files have already been applied, so running it twice does no harm — this is how the structure will be safely updated in future versions.

A deliberate rule: if you choose to use a cloud AI provider instead of a local model, your API key is **never** stored in the database. Only a placeholder note is saved. Real key storage will use the operating system's secure keychain when that part is built.

7. The two pieces of real "thinking" code

Most of the app is currently interface. But two pieces of genuine logic are built, tested, and running on the backend. Both are written as pure functions — meaning they take some numbers in and give an answer out, with no side effects — which makes them easy to test exhaustively.

A. The placement test — working out your English level

There are 20 multiple-choice questions (grammar and vocabulary), arranged from easy to hard and tagged by CEFR level from A1 to C2. The correct answers stay on the server, so the UI never sees them.

Scoring uses a **"ceiling" method**: start at A1 and walk upwards, keeping your level rising as long as you get at least half of that band's questions right. The moment you drop below half, that is your ceiling — and correct answers *above* the ceiling do not count. This stops one lucky guess on a hard question from making someone look like an advanced speaker.

All correct → C2
A1 and A2 right, B1 wrong → A2 (even if C1 answers were right)
Nothing reaches 50% → A1 (the floor)

B. The hardware check — which AI model can this computer run?

Step 3 of onboarding asks you to pick an AI model size. But not every computer can run every size, especially while also running speech recognition and voice synthesis at the same time during a conversation. So the app measures your actual CPU cores and RAM, sends those two numbers to the backend, and gets back a recommendation.

TIER	MODEL	NEEDS	NOTES
Light	qwen2.5-1.5b	6 GB RAM, 2 cores	Runs almost anywhere.
Balanced	gemma-3-4b-it	8 GB RAM, 4 cores	Matches the project's stated target machine.
Heavy	qwen2.5-7b	12 GB RAM, 6 cores	Selectable, but never auto-recommended — too slow for live conversation on a CPU.

If your machine cannot manage even the lightest option, the app says so plainly and switches you to API-key mode instead of letting you pick something that would run badly. You can still override it.

An honest detail worth knowing. Computers report slightly less RAM than they physically have (some is reserved by the system). A "6 GB" machine may report 5.7 GB. The check rounds to the nearest whole gigabyte before comparing, so a machine the screen calls "6 GB" is not rejected by a rule that asks for 6 GB. This was found and fixed by testing on a real 6 GB laptop.

8. What the backend can actually do today

ADDRESS	PURPOSE
GET /health	"Are you awake?" — used at startup.
POST /users	Create your profile (step 1 of onboarding).
GET /users/{id}	Load your profile when the app opens.
PATCH /users/{id}/placement	Save your CEFR level.
PUT /users/{id}/settings	Save AI model choice and daily goals.
POST /users/{id}/companion	Save your companion animal and starting area.
POST /users/{id}/onboarding/complete	Mark onboarding as finished.
GET /placement/questions	Fetch the 20 test questions (without answers).
POST /placement/score	Mark the test and return the estimated level.
POST /engine/assess-hardware	Recommend an AI model size for this computer.

9. What is real and what is a mock-up

This is the most important section for anyone reviewing the project. The app looks complete, but different parts are at very different stages, and the code is honest about which is which.

Fully working, end to end

- The desktop window, the secure connection to the backend, and clean shutdown.
- The whole 5-step onboarding wizard — everything you enter is really saved to the database and really reloaded next time you open the app.
- The 20-question placement test, marked on the server.
- The hardware check and model recommendation, using your real CPU and RAM.
- The folder picker, the window controls, and the dark/light theme.

Built as interface, using sample data

- The dashboard's charts, streak and heat-map.
- The video library, the video player, and the word-lookup panel.
- The bookshelf, the e-reader, highlights, and the text-simplification panel.
- The vocabulary list and word detail pages.

These screens show realistic sample content so the whole app can be navigated and demonstrated. Every one of these files is clearly commented at the top saying the data is temporary and which backend module will replace it.

Not started yet

- The local AI model, speech-to-text, and text-to-speech (no AI library is installed yet).
- Real video playback and real subtitle files.
- The spaced-repetition scheduler, the conversation practice, and the forest.
- Packaging the app into an installer.

Where a screen shows something that would normally come from AI, the interface says so — for example the word-lookup panel is labelled *"not yet configured · offline stub"* rather than pretending a model produced it. This was a deliberate choice: it is better for a demo to be honest about what is simulated than to quietly imply capability that does not exist.

10. Testing

There are **24 automated tests** on the backend, all passing. They cover:

- That requests without the secret token are rejected.
- The complete onboarding sequence: create user → set level → save settings → finish.
- That an API key is never written into the database.
- Every edge case of the placement scoring — all right, all wrong, exactly on the pass line, and the ceiling rule.
- Every edge case of the hardware check, including the real numbers from the development laptop.

Run them with `cd backend && uv run pytest -q`. On the UI side there is no test runner yet; instead, TypeScript checks the whole interface for mistakes at build time, and the build fails if anything is wrong.

11. How to run it

First time only:

```
npm install          # installs the desktop app and UI pieces
uv sync --project backend # sets up Python 3.11 and the server pieces
```

Every time after that:

```
npm run dev
```

That one command starts everything. Press `Ctrl+C` to stop it — the UI, the window, and the Python server all shut down together.

Changes to the interface appear instantly without restarting. Changes to the Electron files or the Python files need a restart, because those are separate programs that load their code once at startup.

To start over from scratch, delete `~/.config/fluencyos/fluencyos.db` and the app will show the onboarding wizard again.

12. Where this goes next

The foundation is deliberately built so that each remaining feature is a matter of replacing sample data with a real source, not restructuring anything:

1. **Media ingestion** — read real video files and subtitles, so the library and player show real content.
2. **The local AI model** — plug in llama.cpp so the word-lookup panel gives real explanations.
3. **The review system** — the spaced-repetition scheduler and flashcards.
4. **Conversation practice** — speech in, AI reply, speech out, and feeding the result back into scheduling.
5. **The forest** — the visual progress system where each word you learn is a growing plant.

The database structure, the API pattern, the store pattern and the design system are already in place, so each of these slots into an existing shape rather than inventing a new one.